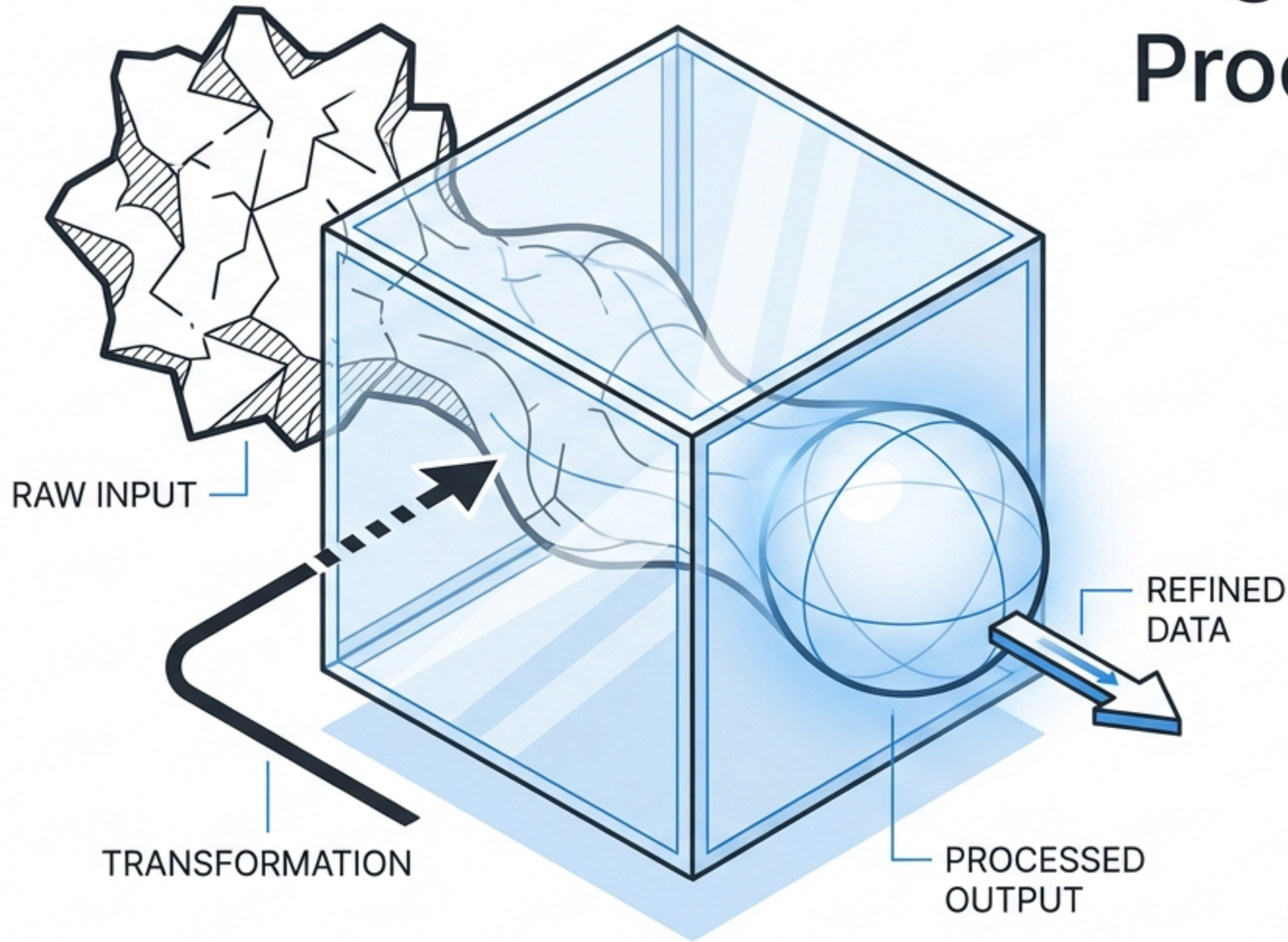


Serverless Image Processing Service

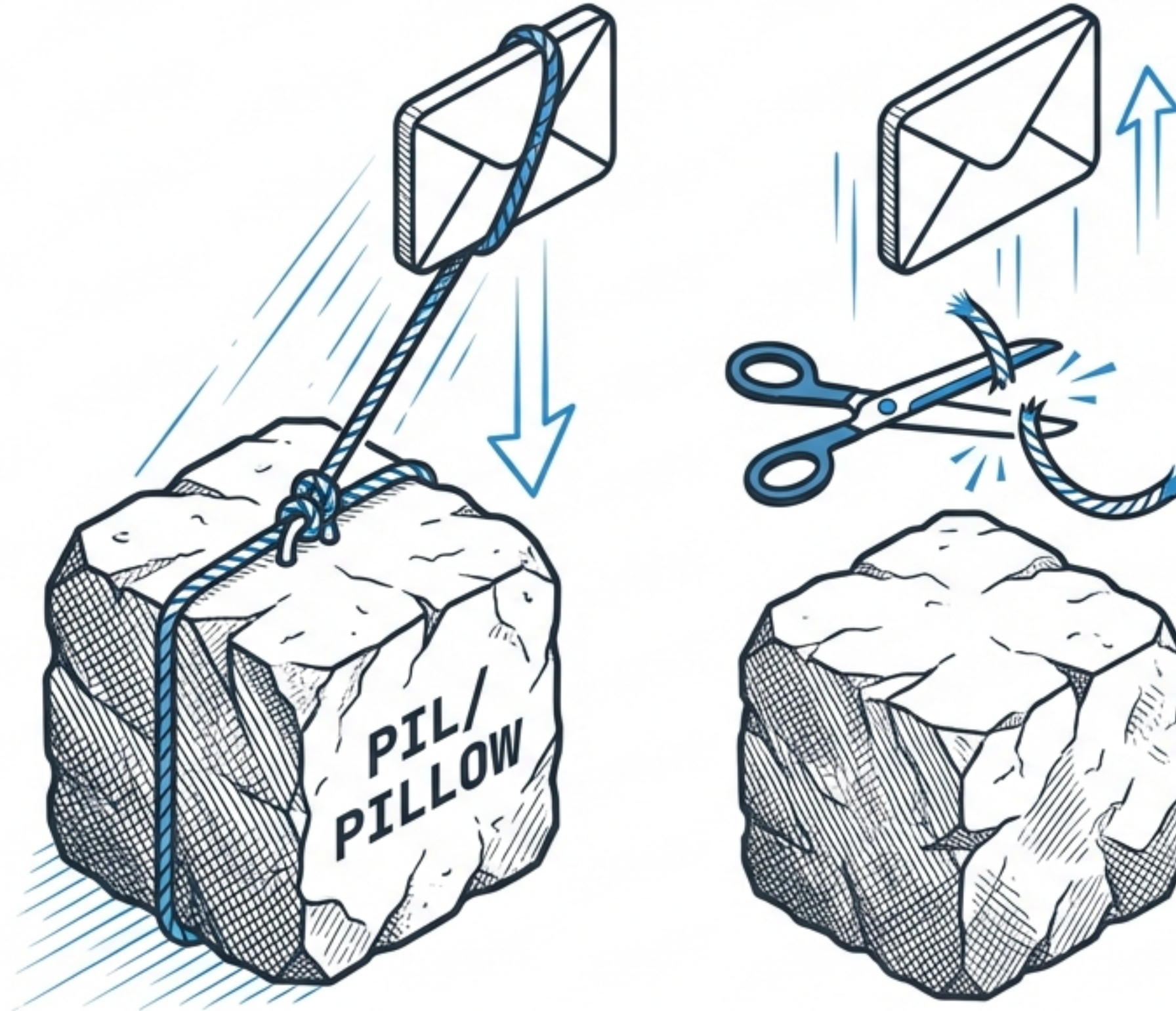
A Stateless, Type-Safe
Architecture for AWS &
Marketplace Distribution.



The Friction of Heavy Dependencies

The Context:

We needed to support file attachments in a Git-backed, graph-based issue tracking system. Images required resizing, compression, and formatting upon upload.



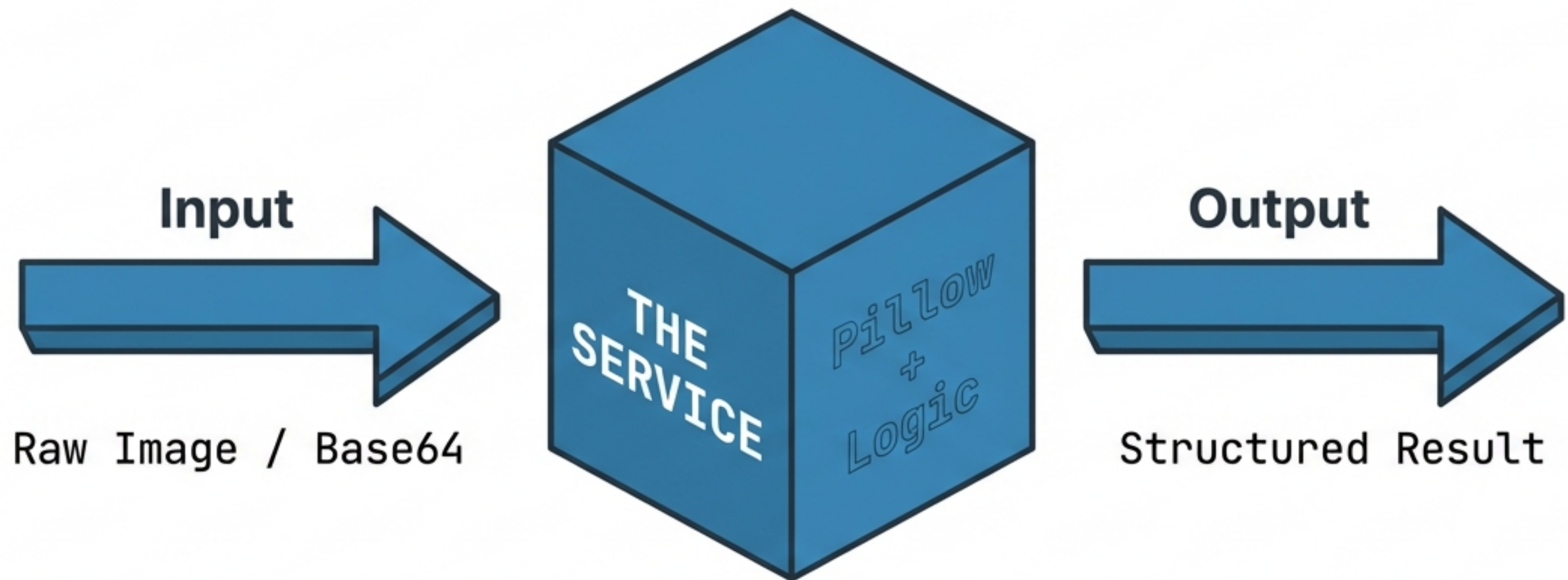
The Challenge:

Embedding heavy image manipulation logic—specifically libraries like Pillow—into every backend service creates massive bloat and dependency hell.

The Insight:

Isolate the weight. Move the heavy lifting to a dedicated, single-purpose location.

A Standalone “Black Box” Solution



A single-purpose microservice designed to be reused across systems and sold independently.

Single Responsibility Principle: One job. **Image In** -> **Transformation** -> **Image Out**.

Core Architectural Manifesto



Stateless

No data is persisted between requests. Each request is self-contained.



Ephemeral

No reliance on local disk or cache. Results are calculated and returned immediately.



Serverless-First

Designed for AWS Lambda burst workloads, adaptable to EC2 for the Marketplace.



Dependency Isolation

Heavy libraries live ONLY inside this service boundaries.



Type-Safe

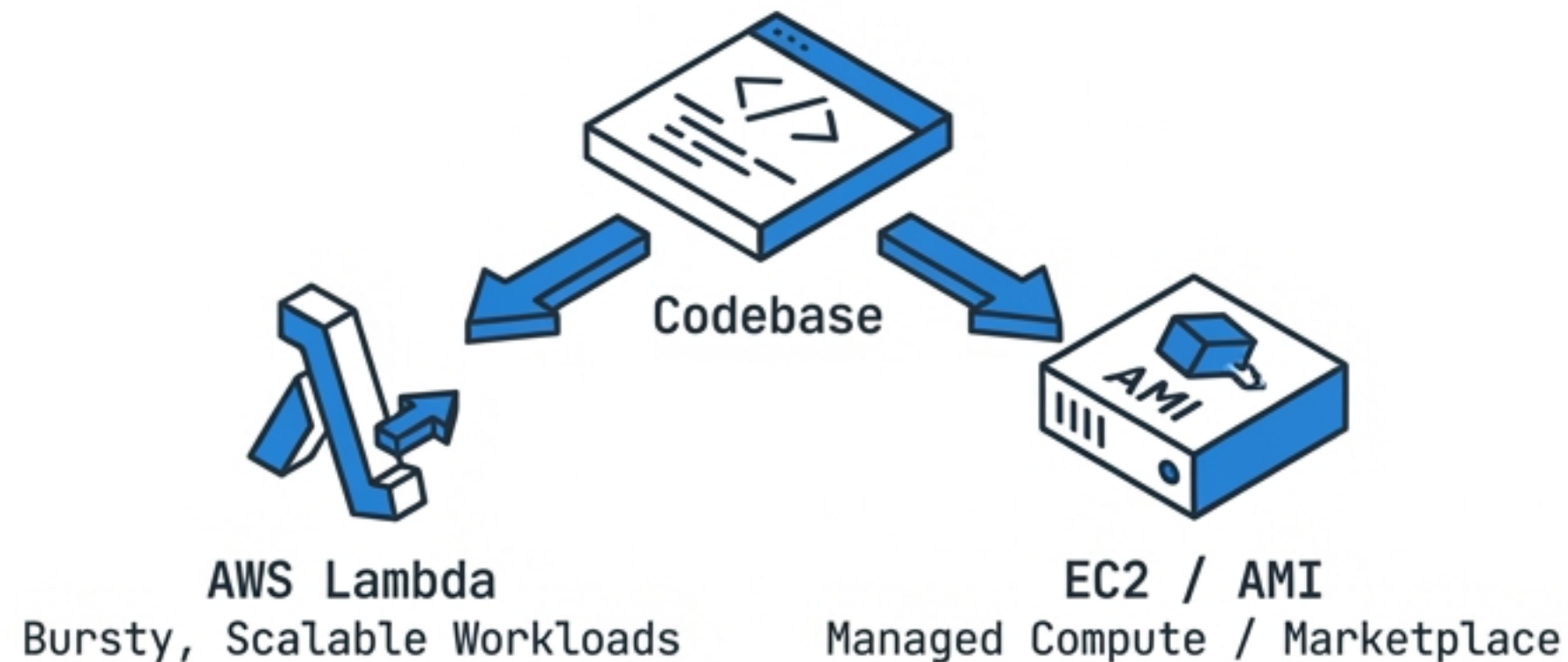
No raw strings. All inputs and outputs use explicit schemas.

Infrastructure & Deployment Strategy

Zero State, Infinite Scale.

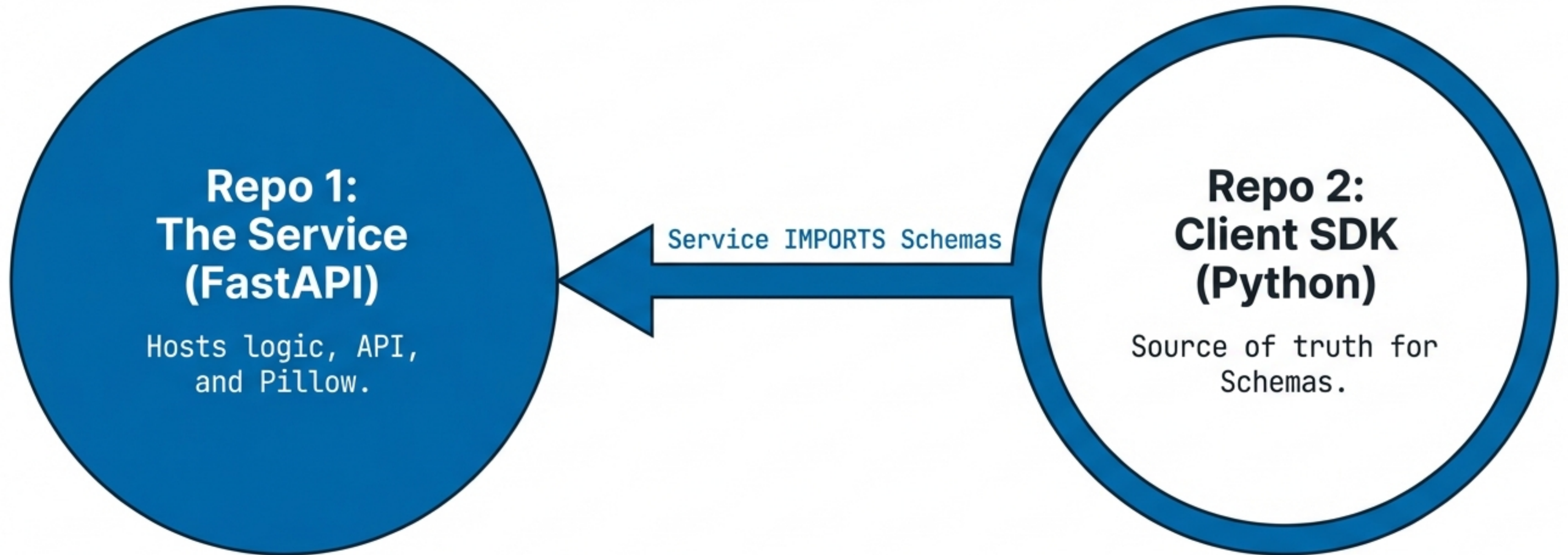


NO STATE STORED



Optimised for serverless execution. Ideally suited for future containerisation, but currently targeting Lambda and AMI artifacts.

The Repository Strategy: Separation of Concerns



The Goal: To enforce schema consistency and guarantee that the client and server speak the exact same language.

The Rule: The Server depends on the Client schemas. The Client NEVER depends on the Server.

Repository Architecture Breakdown

SERVICE REPO



FastAPI Framework



Pillow (PIL) Logic



Validation & Execution



IMPORT: Client Schemas

CLIENT SDK REPO



Request/Response Schemas



Typed Client Methods



API Path Definitions



NO Heavy Dependencies

Result: Schema drift is impossible. Compile-time and runtime guarantees are enforced.

Eradicating "Stringly Typed" Errors


`function(format="jpg", mode="crop")`
Ambiguous String

`function(format=ImageFormat.JPG, mode=ResizeMode.CROP)`

Typed Enum / Constrained Value

Security, correctness, and refactor safety are paramount. We remove ambiguity by banning raw strings for operations and options.

Implementation: All parameters are Enums or Typed Objects using 'osbot_utils' primitives.

Benefit: Invalid requests are caught at the boundary.

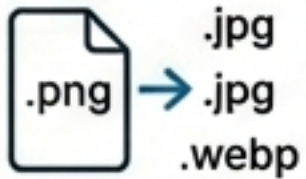
Functional Scope (v0.1.1)

One logical transformation per request.

Transformations



Resize (Width, Height, Aspect)



Format Conversion
(PNG, JPEG, WEBP)



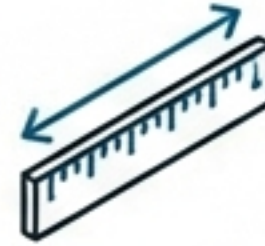
Compression & Quality



Thumbnail Generation

Future Scope: Text-to-Image and OCR.

Metadata Extraction



Dimensions & File Size

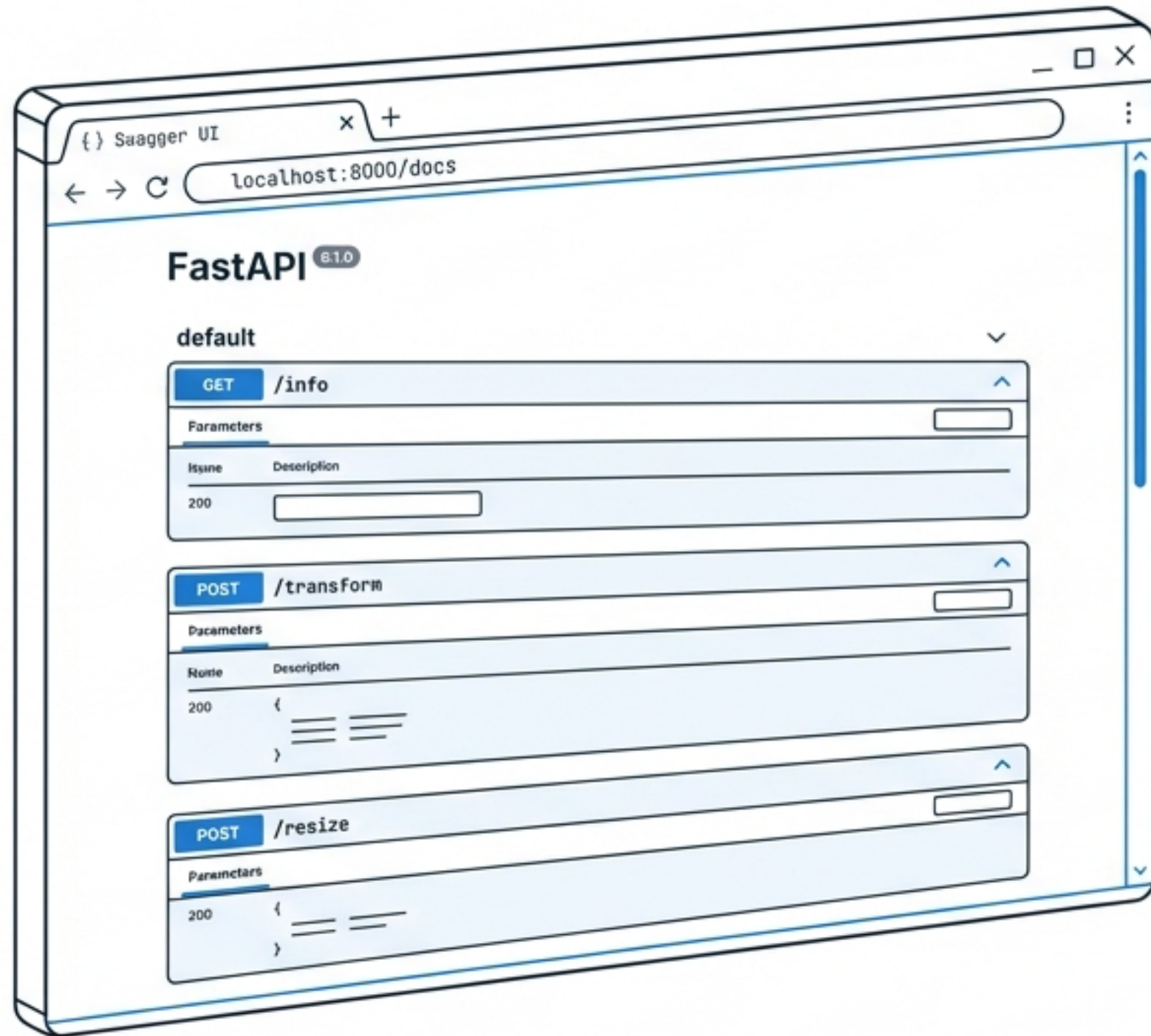


Format & Colour Mode



EXIF Summary

Interface & Developer Experience



API Design:

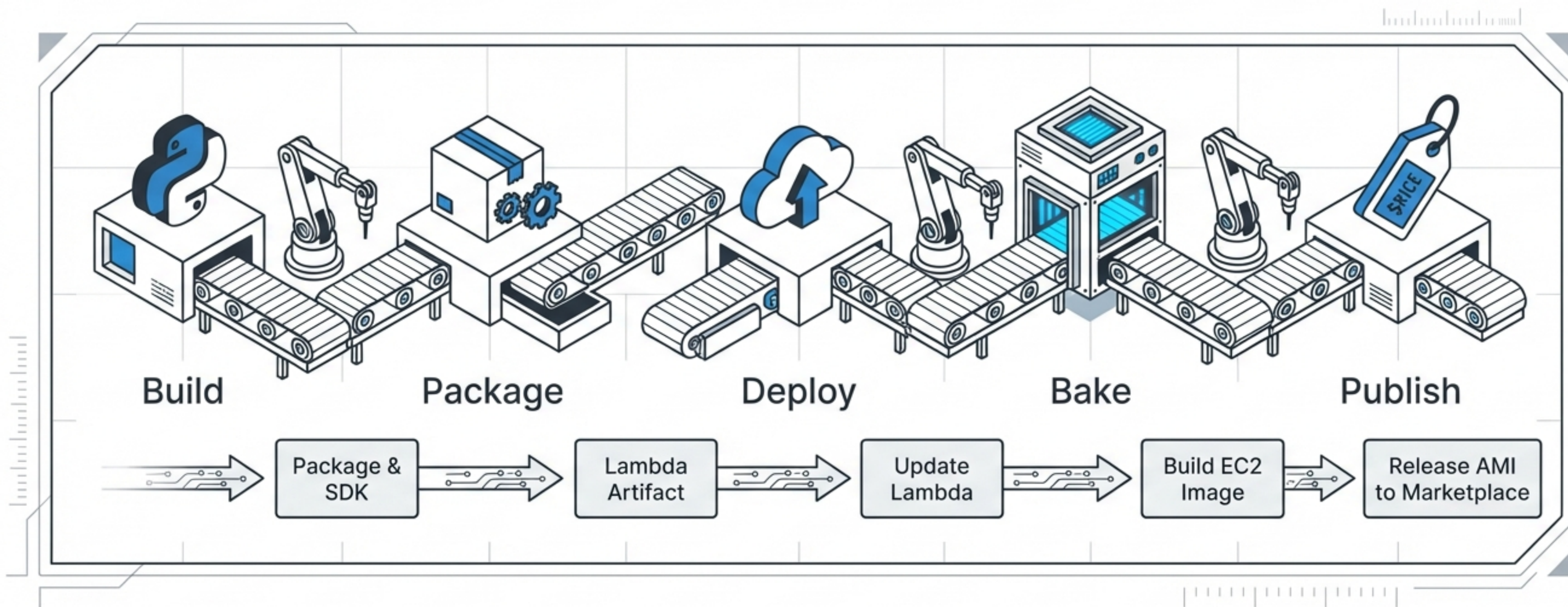
REST API accepting Typed Requests and returning Typed Responses. Supports Multipart Upload and Base64.

Embedded UI Strategy:

- Hosted directly within the FastAPI service
- Includes auto-generated Swagger/OpenAPI docs
- Includes simple HTML/JS UI for manual testing

Benefit: Zero extra deployment needed.

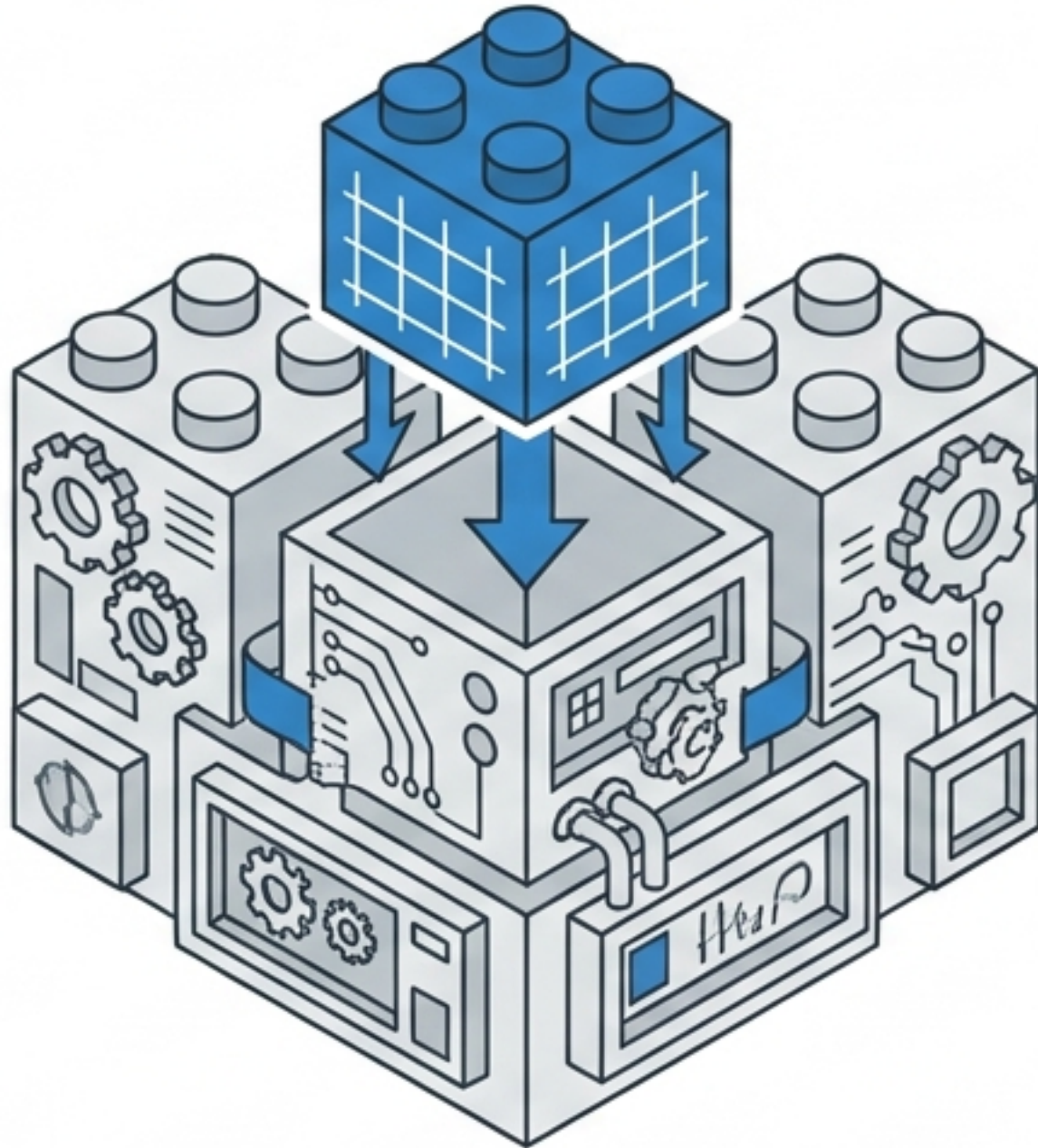
CI/CD & The Build Pipeline



Automated flow supporting simultaneous serverless and VM-based release targets.

The AWS Marketplace Strategy

Why this is a product, not just a utility:



1. Clear Scope

- Easy to understand value proposition.

2. Stateless Architecture

- No customer data retention means low liability and high trust.

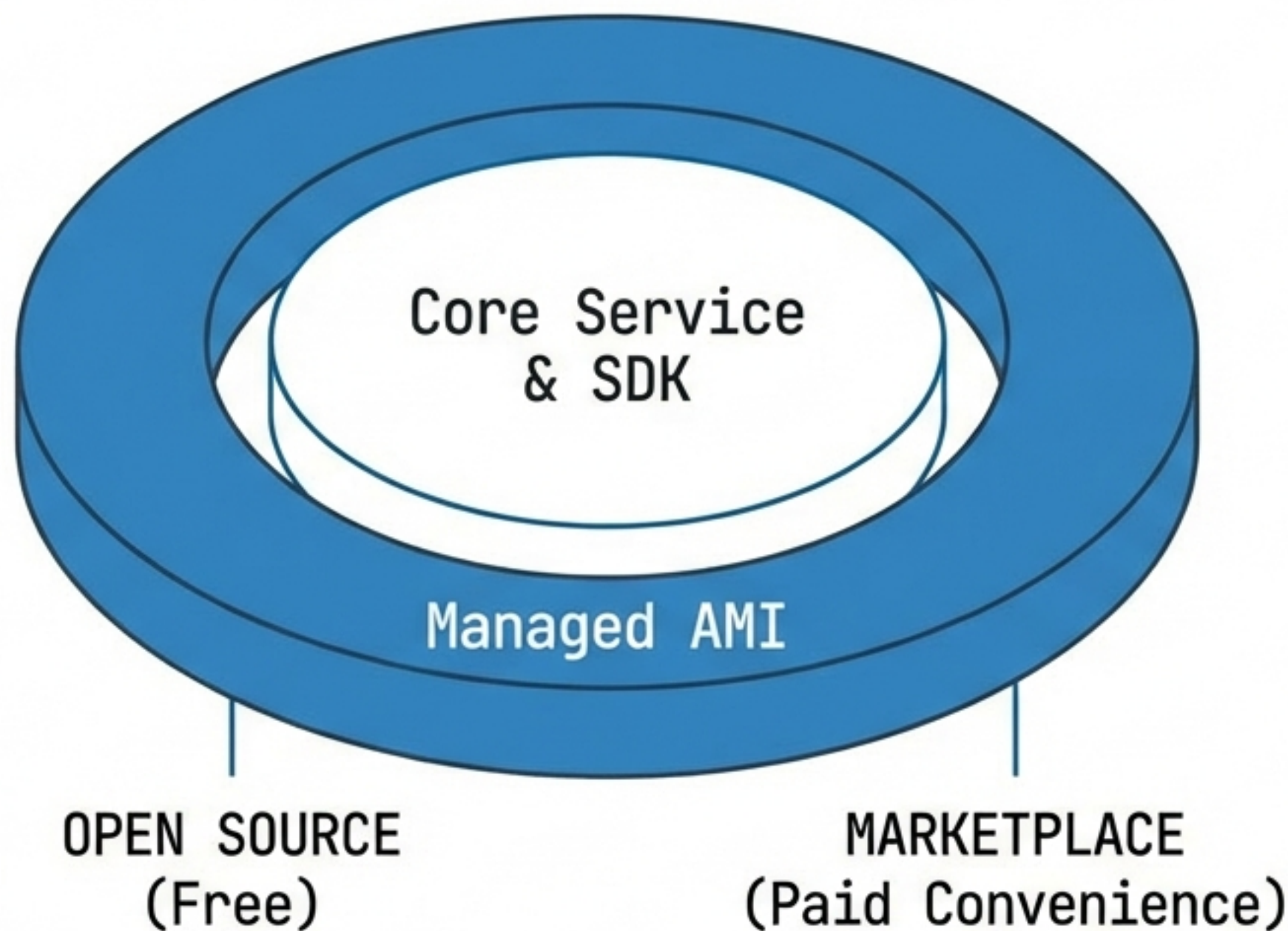
3. Easy Pricing Model

- Per request or per compute hour.

Use Case:

Customers buy this as a building block to add image processing to their systems without maintaining the underlying code.

Open Source Philosophy



Transparency:

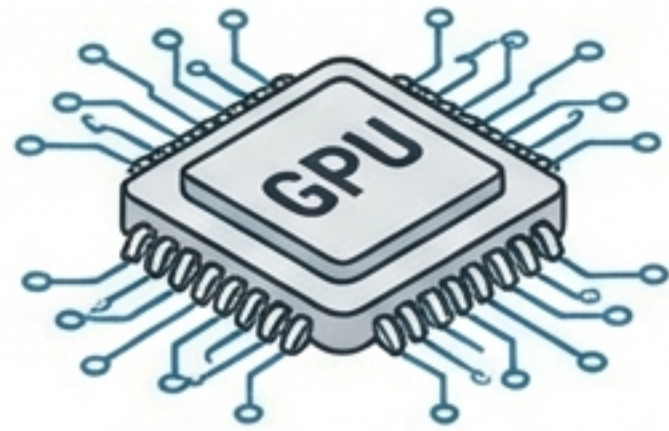
Core logic is fully open to encourage trust and contribution.

Commercial Balance:

The Marketplace AMI bundles the open-source code with deployment convenience and managed infrastructure.

Future Horizons

Capabilities enabled by the current architecture.



Hardware Acceleration

Support for GPU-backed operations.

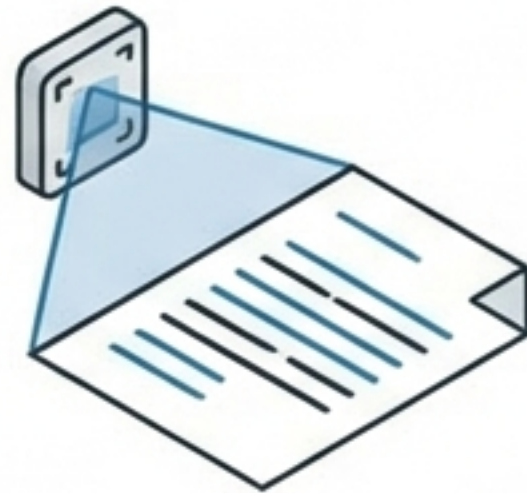


AI Integration

Vision models and enhancement.

OCR

Image-to-Text conversion.



Workflow

Batch processing and Async flows.

Layered extensions that preserve the core stateless principles.

Summary & Key Takeaways



1. Clean & Stateless

A microservice that solves real friction without storing data.



2. Enforced Discipline

Schema consistency via a unique two-repo strategy.



3. Market Ready

Serverless-first deployment with a clear path to AWS Marketplace revenue.



4. Composable Foundation

A reusable base for future media processing services.

Complexity Isolated. Stability Delivered.